

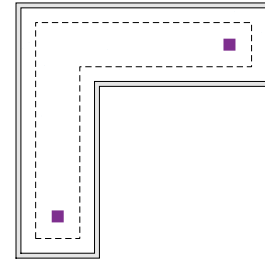


Reviewing the problem

- In Lesson 4.4.3, we set up the geometry and sensor model for an indoor-navigation example.
- In Lesson 4.4.4, we developed the conditional pdfs and importance density needed for the particle filter.
- Now, we're ready to implement the particle filter and examine some results.
- We first define some geometric constants:

```
% Room geometry
w = 0.2; % thickness of wall
W = 10; % width of room
F = 0.7; % L-shape factor (larger value is more extreme)

% Radio-beacon location(s)
NB = 2; % two radio beacons
B = [(1-F)*W/2 W-(1-F)*W/2]; [(1-F)*W/2 W-(1-F)*W/2];
```



Code to draw the room

- The next segment of code draws the room layout and the locations of the radio beacon(s)

```
%% Draw room geometry
figure(1); clf; CL = lines; set(gcf,"Color",[1 1 1]);
axes('Units','normalized','Position',[0 0 1 1]);
x1 = [0 0 W W (1-F)*W (1-F)*W 0];
y1 = [0 W W F*W F*W 0 0];
x0 = [-w -w W+w W+w (1-F)*W+w (1-F)*W+w -w];
y0 = [-w W+w W+w F*W-w F*W-w -w -w];
gray = 0.9;
fill(x0,y0,gray*[1 1 1]); hold on;
fill(x1,y1,'w'); axis square;
set(gca,"Visible","off");
xlim([-2*w W+2*w]); ylim([-2*w W+2*w]);

%% Plot radio beacon(s)
plot(B(1,:),B(2,:), 's', "Color", CL(4,:), "MarkerSize", 20, "LineWidth", 3, ...
      "MarkerFaceColor", CL(4,:));
```



Code to compute and draw vehicle path

- The next segment of code computes and draws the path taken by the vehicle around the room

```
%% Vehicle trajectory: compute true velocity profile (plus initial point)
% f = fraction of min thickness; o = offset because of that
f = 0.2; o = f*(1-F)*W;
% Path = (o,o) ... (o,W-o) ... (W-o,W-o) ... (W-o,(F*W)+o) ...
% ((1-F)*W)-o,(F*W)+o) ... ((1-F)*W)-o,o) ... (o,o)
px = [o o W-o W-o (1-F)*W-o (1-F)*W-o o];
py = [o W-o W-o F*W+o F*W+o o o];

% compute velocity profile of vehicle... update about every dx m
vx = 0; vy = 0; dx = 0.1;
for theS = 1:length(px)-1 % path segment
    len = ceil(sqrt((px(theS+1)-px(theS))^2 + (py(theS+1)-py(theS))^2)/dx);
    % compute (x,y) points along this line segment, then velocities
    pxx = linspace(0,1,len)*(px(theS+1)-px(theS)) + px(theS);
    pyy = linspace(0,1,len)*(py(theS+1)-py(theS)) + py(theS);
    vx = [vx; diff(pxx)']; vy = [vy; diff(pyy)']; %ok<AGROW>
end
pxx = px(1)+cumsum(vx); pyy = py(1)+cumsum(vy); plot(pxx,pyy,'k--')
```



Code to initialize particles

- Particle locations are initialized randomly within room and then plotted as a scatter plot.

```
%% Randomly generate initial particles and plot them
% Room shape = +-----+ Modified shape = +-----+
%           | TOP |           | BOT | TOP |
%           +-----+           +-----+
%           | |           | 0
%           | |
%           +-----+
%
% "Cut off" bottom portion and "glue on" to top portion (via a
% rotation) to generate random points. Then, any random points that have
% "x" coordinate less than zero belong in the bottom portion and get
% translated/rotated into that portion
NP = 100; % number of particles
P = [W; W] - diag([(1+F)*W, (1-F)*W])*rand([2,NP]); % points in mod. shape
P(:,(P(1,:)<0)) = [W; 0] - [0 1; 1 0]*P(:,(P(1,:)<0)); % translate
scatter(P(1,:),P(2,:),30,CL(1,:),"filled");
```



Setting constants

- Constants are defined for initial weights, resampling threshold, radio-propagation model, and algorithm method.

```
%% Set up some constants for the filter
WW = ones(1,NP)/NP; % weights for each particle, random
nEffR = 0.8*NP; % threshold before we resample particles

sS = 0.005; % standard deviation of speed error
sQ = 5*pi/180; % standard deviation of angle error
dT = 1; % time between samples

L1 = 36; % path loss at 1m, in dB
LW = 16; % path loss per wall, in dB
sFade = 1; % standard deviation of log-normal fading in dB

fxxMethod = "sigma-point"; % how to estimate f(x[k] | x[k-1])
```



Main program loop (1)

- The z_k variable is “measured,” including adding noise.
- Noisy speed and angle variables are also “measured.”

```
for theStep = 1:length(vx)
% Make (noisy) zk measurement at the vehicle
loc = [pxx(theStep); pyy(theStep)]; % true vehicle (x,y) location
z = zeros(NB,1);
for theB = 1:NB
dist = norm(loc - B(:,theB));
NW = numWalls(loc,B(:,theB),F,W);
z(theB) = 36+20*log10(dist)+16*NW + sFade*randn(1);
end

% Make (noisy) speed and angle measurement "uk" at the vehicle
sm = norm([vx(theStep); vy(theStep)]) + sS*randn(1);
qm = atan2(vy(theStep), vx(theStep)) + sQ*randn(1);
```



Main program loop (2)

- Convert polar speed/angle to cartesian:

```
switch fxxMethod
case "sigma-point" % Use sigma-point method w. CKF consts from Lesson 3.2.2
    Sw = diag([sS sQ]); WV = sqrt(2)*[Sw -Sw];
    VX = [(sm - WV(1,:)).*cos(qm-WV(2,:)); (sm - WV(1,:)).*sin(qm-WV(2,:))];
    VXhat = sum(VX,2)/4; vxhat = VXhat(1); vyhat = VXhat(2);
    Sa = (VX - VXhat)*(VX - VXhat)'/4;
case "coordinateTx" % Use polar-to-cartesian method from Lesson 2.4.2
    vxhat = sm.*cos(qm)*(1-exp(-sQ^2)+exp(-sQ^2/2));
    vyhat = sm.*sin(qm)*(1-exp(-sQ^2)+exp(-sQ^2/2));
    S12a = sin(qm).*cos(qm).*exp(-4*sQ^2).*(sS^2+(sm.^2+sS^2)*(1-exp(sQ^2)));
    Sa = [sm.^2*exp(-2*sQ^2).*((cos(qm)).^2.*(cosh(2*sQ^2)-cosh(sQ^2)) ...
        +(sin(qm)).^2.*(sinh(2*sQ^2)-sinh(sQ^2))) ...
        +sS^2*exp(-2*sQ^2).*((cos(qm)).^2.*(2*cosh(2*sQ^2)-cosh(sQ^2)) ...
        +(sin(qm)).^2.*(2*sinh(2*sQ^2)-sinh(sQ^2))), S12a; S12a, ...
        sm.^2*exp(-2*sQ^2).*((sin(qm)).^2.*(cosh(2*sQ^2)-cosh(sQ^2)) ...
        +(cos(qm)).^2.*(sinh(2*sQ^2)-sinh(sQ^2))) ...
        +sS^2*exp(-2*sQ^2).*((sin(qm)).^2.*(2*cosh(2*sQ^2)-cosh(sQ^2)) ...
        +(cos(qm)).^2.*(2*sinh(2*sQ^2)-sinh(sQ^2)))]];
end
```



Main program loop (3)

- Update the particles:

```
% Update particles
for theP = 1:NP
    % Randomly generate new particle location... sample q(xk|xk-1)
    P(:,theP) = P(:,theP) + [vxhat; vyhat] + chol(Sa,"lower")*randn(2,1);

    % Choose to constrain if estimates hit main outside walls
    P(1,theP) = max(0,P(1,theP)); P(1,theP) = min(W,P(1,theP));
    P(2,theP) = max(0,P(2,theP)); P(2,theP) = min(W,P(2,theP));

    % Compute f(zk|xk)
    fz = 1; % initialize... fz = product of each measurement prob.
    for theB = 1:NB
        dist = norm(P(:,theP) - B(:,theB));
        NW = numWalls(P(:,theP),B(:,theB),F,W);
        fz = fz * normpdf(z(theB),L1+20*log10(dist)+LW*NW,sFade);
    end
    WW(theP) = WW(theP) * fz; % compute unnormalized weight
end
WW = WW/sum(WW); % normalize the weights
```



Main program loop (4)

- Resample particles if needed, and plot present locations.

```
% Maybe resample the particles
nEff = 1/sum(WW.^2); % Actual number of "effective" particles
if nEff<nEffR % check to see if this is below the "resample" threshold
    iPick = 1; Pnew = zeros(2,NP); c = cumsum(WW); % initialize
    threshold = rand(1)/NP;
    for cp=1:NP
        while threshold>c(iPick) && iPick<NP
            iPick = iPick + 1;
        end
        Pnew(:,cp) = P(:,iPick);
        threshold = threshold + 1/NP;
    end
    P = Pnew; % the new set of particles
    WW = ones(1,NP)/NP; % the new weights
end

scatter(P(1,:),P(2,:),100*WW+1,CL(1+mod(theStep,255),:),"filled");
end
```



A function to compute number of wall crossings

- Compute the number of walls between points p1 and p2.

```
function NW = numWalls(p1,p2,F,W)
    NW = 0; % default is zero wall crossings
    % Calculate slope (m) and intercept (b) of (infinite-length) line passing
    % through this particle and this radio beacon
    m = (p1(2)-p2(2))/(p1(1)-p2(1)); b = p2(2)-m*p2(1);

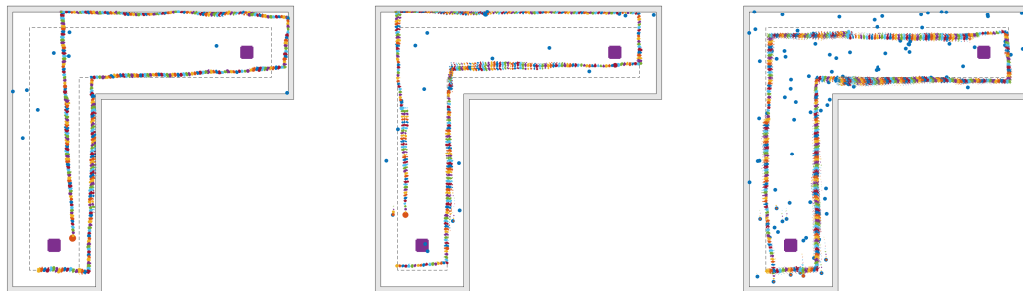
    % Check to see if line *segment* b/w p1 and p2 passes through horizontal wall
    x = (F*W-b)/m; % "x" location where line passes through y=F*W
    condX = x > min(p1(1),p2(1)) && x < max(p1(1),p2(1));
    if condX && x > (1-F)*W && x < W % if intersection point is between
        NW = NW + 1; % "x" endpoints of both lines
    end

    % Check to see if line *segment* b/w p1 and p2 passes through vertical wall
    y = m*(1-F)*W+b; % "y" location where line passes through x=(1-F)*W
    condY = y > min(p1(2),p2(2)) && y < max(p1(2),p2(2));
    if condY && y > 0 && y < F*W % if intersection point is between
        NW = NW + 1; % "y" endpoints of both lines
    end
end
```



Indoor navigation results

- Figures show initial state, and trajectories leading to end state of simulations using 10, 20, and 100 particles.
- Different colors show particles at different timesteps.
- Using more particles leads to better initial estimates and position estimates; all simulations end with particles clustered very close to true vehicle position.



Summary

- You have now learned how to implement a particle filter in Octave for the application of indoor navigation.
- Most of this code is specific to setting up the parameters of the indoor space, the pdfs required by the particle filter that apply to this problem, and so forth.
- The code that is specific to the particle-filter implementation itself is essentially unchanged from what you have seen before.
- So, you have learned that the real challenge to applying the particle filter to the indoor-navigation application has to do with understanding and modeling the application itself, not to implementing a particle filter in general.
- I was very pleased with the quality of the navigation estimates; I imagine that they could be improved by using more radio beacons, more particles, etc.